Java Silver（SE 17・1Z0-825-JPN）　配列とコレクション　#04

配列と ArrayList

長さが決まった配列と、伸び縮みする ArrayList ― 作り方、添字、コピー、Arrays、List.of と Arrays.asList

各ページの下に、根拠にした Oracle の資料（JLS・Java SE 17 API）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 配列の宣言・初期化・操作と ArrayList

1 配列の宣言と生成
new int[3]・{1, 2, 3}・既定値・添字・length・例外

2 多次元配列と配列の操作
非対称な配列、拡張 for、clone と浅いコピー、Arrays クラス

3 ArrayList
ジェネリクス、add・get・set・remove、ConcurrentModificationException

4 変更できないリストとラムダ式
List.of と Arrays.asList のちがい、removeIf・forEach

	配列	ArrayList
長さ	作ったら**変わらない**	add・remove で**変わる**
要素の型	基本データ型も参照型も	参照型だけ（int は Integer）
要素数	`a.length`	`list.size()`
取り出し	`a[i]`	`list.get(i)`
範囲外	ArrayIndexOutOfBounds…	IndexOutOfBounds…
equals	同じ配列か（== と同じ）	同じ順に同じ要素か
表示	`[I@1dbd16a6` の形	`[a, b]`

芯：配列の変数・リストの変数が持つのは**参照**。代入はオブジェクトを共有するだけで、コピーにはならない。

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JLS 10 Arrays（Once an array object is created, its length never changes）／Java SE 17 API ArrayList（Resizable-array implementation of the List interface）

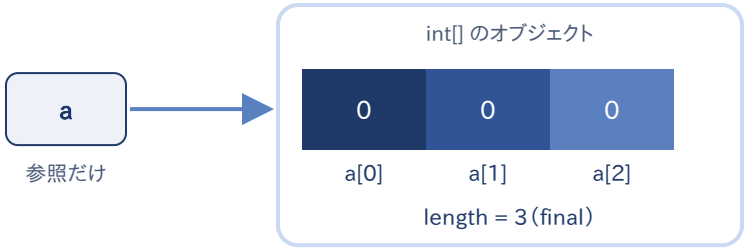
第1章

配列の宣言と生成

宣言だけでは配列はない。new か {…} で作って、はじめて要素ができる。

配列はオブジェクト、変数は参照

int[] a = new int[3]; のあと（Claude が作成）



要素は既定値で埋まる。長さはあとから変えられない

要素の型	既定値（JDK 17 の表示）
byte・short・int・long	0
float・double	0.0
boolean	false
char	コード 0 の文字（`'\u0000'`）
参照型（String など）	null

既定値があるのは配列の要素とフィールド。ローカル変数には既定値がなく、代入前に使うとコンパイルエラー。

配列はオブジェクト：Object 型の変数に入り、Object のメソッドを呼べる。長さは型の一部ではないので、`int[]` の変数は長さ 3 の配列も長さ 100 の配列も指せる。

出典：JLS 10 Arrays（arrays are objects … may be assigned to variables of type Object／10.1 An array's length is not part of its type／10.2 Declaring a variable of array type does not create an array object）／JLS 4.12.5 Initial Values of Variables／結果は JDK 17 で確かめたもの

配列の作り方 ― 書ける形・書けない形

書き方（main の中）	結果（JDK 17）
`int[] a = new int[3];`	OK（要素は 0）
`int[] a = {1, 2, 3};`	OK（長さ 3）
`int[] a = {1, 2, 3,};`	OK（最後のカンマは無視）
`int a[] = {1, 2};`	OK（C 言語風）
`a = new int[]{1, 2, 3};`	OK（あとから作り直す）
`a = {1, 2, 3};`	illegal start of expression
`new int[3]{1, 2, 3}`	both dimension expression and initialization is illegal
`var b = {1, 2};`	cannot infer type
`new int[n]`（n は long）	possible lossy conversion

[] の位置で型が決まる（Claude が作成）

宣言	それぞれの型
`int[] a, b;`	a:int[]　b:int[]
`int[] a, b[];`	a:int[]　b:**int[][]**
`int a, b[];`	a:**int**　b:int[]
`int[] a[];`	a:int[][]

決まり：型の後ろの [] は**全部の変数**に、名前の後ろの [] は**その変数だけ**に付く。両方あれば足し算。

{…} だけで作れるのは宣言と同時のときだけ。あとから代入するときやメソッドに渡すときは `new int[]{…}`。

出典：JLS 10.2（例 10.2-2：int a, b[], c[][]; is equivalent to … int a; int[] b; int[][] c;）・10.6 Array Initializers（A trailing comma may appear after the last expression）／JLS 15.10.1（ArrayCreationExpression の構文、The promoted type must be int）／結果は JDK 17 で確かめたもの

添字・長さ・例外

```
int[] a = {10, 20, 30};
System.out.println(a[0] + a[a.length - 1]);  // 40
System.out.println(a[3]);
```

```
40
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
 Index 3 out of bounds for length 3
```

添字は 0〜length−1。範囲外はコンパイルは通り、**実行時**に例外。

数え方	範囲外の例外
配列 `a.length`	ArrayIndexOutOfBoundsException
String `s.length()`	StringIndexOutOfBoundsException
ArrayList `list.size()`	IndexOutOfBoundsException

コード	結果（JDK 17）
`new int[0]`	OK（長さ 0 の配列）
`new int[n]`（n は −1）	実行時 NegativeArraySizeException
`a[-1]`	実行時 ArrayIndexOutOfBoundsException
`a.length`	要素数（フィールド）
`a.length()`・`a.size()`	cannot find symbol

出典：JLS 10.4 Array Access（All arrays are 0-origin／All array accesses are checked at run time … ArrayIndexOutOfBoundsException）・10.7（The public final field length）／JLS 15.10.2（less than zero, then a NegativeArraySizeException is thrown）／結果は JDK 17 で確かめたもの

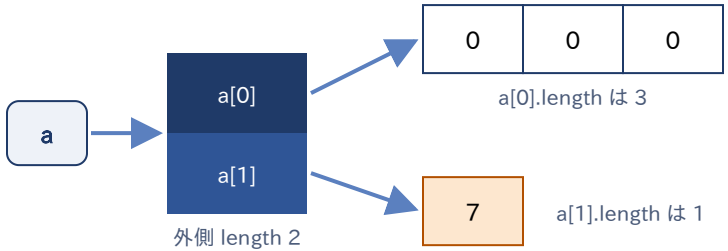
第2章

多次元配列と配列の操作

配列の配列。内側の長さはそろわなくてよい。コピーは浅い。

多次元配列は「配列の配列」

int[][] a = new int[2][]; a[0] = new int[3]; a[1] = new int[]{7};（Claude が作成）



new int[2][] の直後は a[0]・a[1] とも null

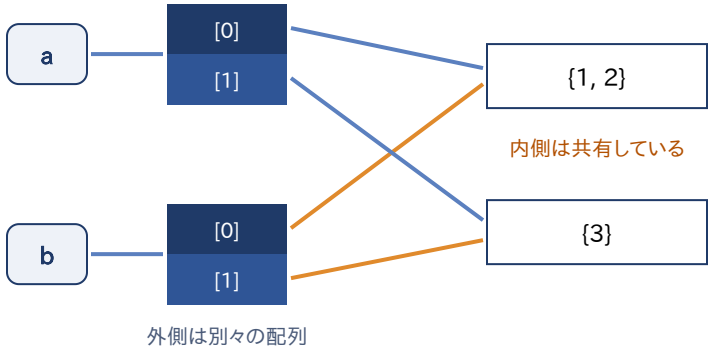
コード	結果（JDK 17）
`new int[2][3]`	長さ 3 の int[] を 2 つ
`new int[2][]`	外側だけ。中は null
`new int[][3]`	']' expected
`{{1, 2, 3}, {4, 5}, {6}}`	内側の長さ 3・2・1
上の `a[1].length`・`a[2][0]`	2・6
null の行に `a[0][0]`	NullPointerException

```
for (int[] row : a) {        // 外側の要素は int[]
  for (int x : row) sum += x;
}
```

出典：JLS 15.10.2（A multidimensional array need not have arrays of the same length at each level／例 15.10.2-2 new float[3][3] は new float[3][] とループに同じ）／JLS 10.6 例 10.6-1（null の行で NullPointerException）／結果は JDK 17 で確かめたもの

代入・clone・浅いコピー

int[][] b = a.clone(); のあと（Claude が作成）



操作（a は {{1, 2}, {3}}）	a から見えるか
`int[][] c = a; c[1] = …`	見える（同じ配列）
`b[0][0] = 9;`	見える（内側を共有）
`b[1] = new int[]{7};`	見えない（外側は別）
`a == b`	false

1 次元の clone は中身ごと別。`int[] c = a.clone(); c[1] = 8;` は a を変えない。**多次元は外側だけ新しい（浅いコピー）。**

拡張 for の変数はコピー:`for (int x : a) x = x * 10;` は a を変えない。

出典：JLS 10.7 Array Members（A clone of a multidimensional array is shallow, which is to say that it creates only a single new array. Subarrays are shared／例 10.7-1・10.7-2）／JLS 14.14.2／結果は JDK 17 で確かめたもの

ArrayStoreException ― 配列の型は実行時に調べる

```
Object[] o = new String[2];   // OK：String[] は Object[] に入る
o[0] = "a";                   // OK
o[1] = Integer.valueOf(1);    // コンパイルは通る
```

```
Exception in thread "main" java.lang.ArrayStoreException:
 java.lang.Integer
```

コンパイラが見るのは変数の型（Object[]）。実際の配列は String[] なので、JVM が代入のたびに要素の型を調べて止める。

代入	結果（JDK 17）
`Object[] o = new String[2];`	OK（配列は共変）
`o[1] = 1;`（上の o）	実行時 ArrayStoreException
`List<Object> l = new ArrayList<String>();`	コンパイルエラー（incompatible types）
`int[] x = new long[1];`	コンパイルエラー

配列とジェネリクスの差：配列は「String[] は Object[] の一種」として代入でき、まちがいは**実行時**に見つかる。List<String> は List<Object> に代入できず、**コンパイル時**に止まる。

出典：JLS 10.5 Array Store Exception（an assignment to a component of the array is checked at run time … ArrayStoreException is thrown／例 10.5-1）・10.2（v can hold a reference to an instance of any array type B[], provided B can be assigned to A）／結果は JDK 17 で確かめたもの

Arrays クラスのメソッド

コード（a は {3, 1, 2}）	結果（JDK 17）
`System.out.println(a)`	`[I@1dbd16a6` の形
`Arrays.toString(a)`	"[3, 1, 2]"
`Arrays.sort(a)`（戻り値 void）	a が [1, 2, 3] に
`Arrays.fill(b, 7)`（b は長さ 4）	[7, 7, 7, 7]
`Arrays.copyOf(b, 6)`	[7, 7, 7, 7, 0, 0]
`a.equals(a.clone())`	false（== と同じ）
`Arrays.equals(a, a.clone())`	true（中身を比べる）

[I は「int の配列」を表す型の記号。@ のあとの数字は実行ごとに変わりうる。

文字列の配列を sort すると（文字コード順）

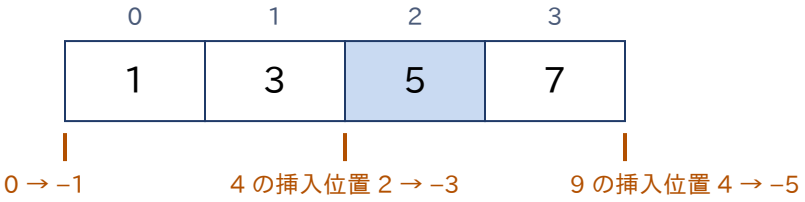
```
String[] s = {"b", "B", "a", "10", "9"};
Arrays.sort(s);
// [10, 9, B, a, b]
```

順番	理由
数字	'0'〜'9' は 48〜57。"10" は先頭 '1' で "9" より前
大文字	'A'〜'Z' は 65〜90
小文字	'a'〜'z' は 97〜122

sort は配列そのものを並べかえる。新しい配列は返さない。

出典：Java SE 17 API Arrays（toString・sort（into ascending order / natural ordering）・fill・copyOf（padding with zeros）・equals）／JLS 10.8 例 10.8-1（"[I" is the run-time type signature）／結果は JDK 17 で確かめたもの

binarySearch・compare・mismatch



5 は添字 2 にある → 2

見つからないとき：−(挿入位置) − 1

挿入位置 ＝ その値より大きい最初の添字

−1 するので「見つかった（0 以上）」と区別できる

compare（辞書順）	結果
`compare({1,2,3}, {1,2,4})`	−1（3 < 4）
`compare({1,2,3}, {1,2})`	1（長いほうが後）
`compare(a, a.clone())`	0

mismatch（最初にちがう添字）	結果
`mismatch({1,2,3}, {1,2,4})`	2
`mismatch({1,2,3}, {1,2})`	2（短いほうの長さ）
`mismatch(a, a.clone())`	−1（全部同じ）

binarySearch は並べかえ済みが前提。並べかえていない配列の結果は不定（API：the results are undefined）。

出典：Java SE 17 API Arrays binarySearch（The array must be sorted … otherwise, (-(insertion point) - 1) … the index of the first element greater than the key, or a.length）／compare(int[], int[])（Compares two int arrays lexicographically）／mismatch(int[], int[])（If one array is a proper prefix of the other then the returned index is the length of the smaller array）／結果は JDK 17 で確かめたもの

第3章

ArrayList

中に配列を持ち、足りなくなったら自分で大きくするリスト。

宣言の形 ― ジェネリクスとダイヤモンド

宣言（java.util.* は import 済み）	結果（JDK 17）
`List<String> l = new ArrayList<>();`	OK（ふつうの形）
`List<Integer> l = new ArrayList<>(100);`	OK（容量 100・size 0）
`List<int> l = new ArrayList<>();`	unexpected type
`List<String> l = new List<>();`	List is abstract; cannot be instantiated
`List<Object> l = new ArrayList<String>();`	incompatible types

型引数は参照型だけ。int は Integer にして入れる。`list.add(10)` はオートボクシングで Integer になる。

var と <> を組み合わせると

```
var list = new ArrayList<>();  // ArrayList<Object>
list.add("a");
list.add(1);                       // どちらも入る
String s = list.get(0);        // コンパイルエラー
```

```
error: incompatible types: Object cannot be converted to String
```

容量と size は別。容量は中の配列の大きさ（引数なしは 10）。size は入っている要素の数。`new ArrayList<>(10)` のあと `set(0, "a")` は IndexOutOfBoundsException。

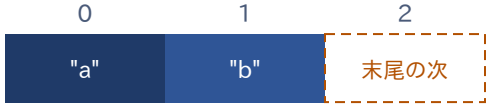
出典：Java SE 17 API ArrayList（ArrayList() Constructs an empty list with an initial capacity of ten）／JLS 4.5.1（Type arguments may be either reference types or wildcards）／JLS 14.4.1（var）／結果は JDK 17 で確かめたもの

add・set・remove をたどる

呼び出し	list の中身・戻り値
`add("a"); add("b");`	`[a, b]`
`add(1, "c")`	`[a, c, b]`（b を後ろへずらす）
`add("d")`	`[a, c, b, d]`　戻り値 true
`set(1, "X")`	`[a, X, b, d]`　戻り値 **"c"**（前の値）
`remove(0)`	`[X, b, d]`　戻り値 "a"
`remove("z")`	変わらない　戻り値 false
`indexOf("d")`・`indexOf("z")`	2・−1

表の流れは Claude が手で追った例（各メソッドの動きは JDK 17 で確かめた）。

size() が 2 のとき使える添字（Claude が作成）



get・set・remove(int)：0〜1

add(i, e)：0〜2（size() と同じなら末尾に足す）

> **remove の 2 つの形：**`List<Integer>` で `remove(1)` は**添字 1** を消す（int をとる remove(int) が先に選ばれる）。値 1 を消すなら `remove(Integer.valueOf(1))`。

出典：Java SE 17 API ArrayList add(int, E)（Shifts the element currently at that position … to the right／index > size() で例外）・set（Returns: the element previously at the specified position）・remove(int)・remove(Object)（If the list does not contain the element, it is unchanged）・indexOf／結果は JDK 17 で確かめたもの

回しながら消すと ConcurrentModificationException

NG:拡張 for の中でリストの remove

```
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
for (String s : list) {
  if (s.equals("a")) list.remove(s);
}
```

```
Exception in thread "main" java.util.ConcurrentModificationException
```

OK:removeIf か Iterator の remove

```
list.removeIf(s -> s.equals("a"));       // [b, c]

Iterator<String> it = list.iterator();
while (it.hasNext()) {
  if (it.next().equals("a")) it.remove(); // OK
}
```

用語	中身
イテレータ	要素を 1 つずつ取り出す係。拡張 for は中でこれを使う
構造の変更	add・remove・clear など要素数が変わる操作。set は含まない
fail-fast	イテレータ自身以外による構造の変更に気づいたら、すぐ例外で止まる
best-effort	必ず止まる保証はない。例外を当てにしたコードは書かない

本番での判断:拡張 for の中で list.add・list.remove を呼ぶコード → ConcurrentModificationException を選ぶ。安全な書き方は removeIf・Iterator.remove。

出典:Java SE 17 API ArrayList(The iterators returned by this class's iterator and listIterator methods are fail-fast: if the list is structurally modified at any time after the iterator is created, in any way except through the iterator's own remove or add methods, the iterator will throw a ConcurrentModificationException／Fail-fast iterators throw ConcurrentModificationException on a best-effort basis)／結果は JDK 17 で確かめたもの

第4章

変更できないリストとラムダ式

どれも List 型。でも、できる操作がちがう。

ArrayList・Arrays.asList・List.of のちがい

作り方	add・remove	set	null を入れる
`new ArrayList<>()`	できる	できる	入る
`Arrays.asList(arr)`	UnsupportedOperationException	できる（元の配列も変わる）	入る
`List.of(…)`	UnsupportedOperationException	UnsupportedOperationException	NullPointerException

Arrays.asList は配列と中身を共有する（Claude が作成）

```
String[] arr = {"a", "b", "c"};
List<String> list = Arrays.asList(arr);
list.set(0, "X");  // arr[0] も "X"
arr[2] = "Z";       // list からも見える
System.out.println(list + " " + arr[0]);
list.add("d");
```

```
[X, b, Z] X
java.lang.UnsupportedOperationException
```

どれもコンパイルは通る。List インタフェースには add も set もあるので、できるかどうかは**実行時**に分かる。

変更できるリストにしたいとき:`new ArrayList<>(List.of("a", "b"))` のようにコピーする。

要素の中身は別の話: List.of の要素が StringBuilder なら `list.get(0).append("b")` はできる（リストの並びは変えていない）。

出典:Java SE 17 API List Unmodifiable Lists（Calling any mutator method on the List will always cause UnsupportedOperationException to be thrown／They disallow null elements／if the contained elements are themselves mutable, this may cause the List's contents to appear to change）／API Arrays.asList（Returns a fixed-size list backed by the specified array）／結果は JDK 17 で確かめたもの

ラムダ式で removeIf・forEach

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));
list.removeIf(n -> n % 2 == 0);           // 偶数を消す
list.forEach(n -> System.out.print(n + " "));
```

```
1 3 5
```

ラムダ式の形（Claude が作成）



書き方	意味
`n -> n * 2`	引数 1 つ。かっこ省略可
`(a, b) -> a + b`	引数 2 つはかっこが必要
`() -> 42`	引数なし
`n -> { return n > 0; }`	{ } なら return と ; が必要

メソッド	動き
removeIf	条件が true の要素を**リスト自身から**消す。走査中でも安全
forEach	全要素に順に処理。戻り値 void

出典：Java SE 17 API Collection.removeIf（Removes all of the elements of this collection that satisfy the given predicate）／API Iterable.forEach／JLS 15.27 Lambda Expressions／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
配列の宣言がコンパイルできるか	長さと {…} の同時はダメ。{…} だけは宣言と同時のみ。多次元は左の [] から長さを書く
何も入れていない要素の表示	既定値：0・0.0・false・null
要素数の書き方	配列 length、String length()、ArrayList size()
範囲外・負の長さ	コンパイルは通る。実行時に ArrayIndexOutOfBounds・NegativeArraySize
コピーしたあとの表示	代入は共有。clone は別の配列だが、多次元の内側は共有（浅いコピー）
配列の比較・表示	equals は ==。中身は Arrays.equals・Arrays.toString
ArrayList の操作の結果	add(i, e) は挿入。set は前の値を返す。remove(int) と remove(Object) を見分ける
リストを変更できるか	List.of は全部ダメ。asList は set だけ OK。どれも実行時の例外

出典：このPDFの各ページの資料

まとめ

配列の宣言と生成
長さは固定。要素は既定値。添字は 0〜length−1

多次元配列と操作
配列の配列。clone は浅い。Arrays で表示・並べかえ・探索・比較

ArrayList
型引数は参照型。add・set・remove の戻り値と添字の範囲

変更できないリスト
List.of は変更不可・null 不可。asList は大きさ固定

式	結果
`new String[2]` の要素	null
`int[] a, b[];` の b	int[][]
`binarySearch({1,3,5,7}, 4)`	−3
[10, 20, 30] で `remove(1)`	[10, 30]
`List.of("a").add("b")`	UnsupportedOperationException

次は #05 演算子。インクリメント、優先順位、== と equals、論理演算の短絡評価へ進む。

出典：このPDFの各ページの資料